Universidad Nacional
Escuela de Informática y Computación

DOCUMENTO DE ARQUITECTURA · LESCOCR

Proyecto final · EIF509 Desarrollo de Aplicaciones Basadas en Web · II Ciclo 2026

Integrantes	Derrek Adrián Ureña Solís y José Arrieta Sancho
Profesor	Elberth Adrián Garro Sánchez
Sistema en línea	SPA: lescocr-spa.onrender.com · API: lescocr-api.onrender.com · Swagger UI: lescocr-api.onrender.com/swagger-ui.html
Repositorio	github.com/xDantes/Lab1-Desarrollo_Apps_Basadas_web

1. Descripción del sistema

LescoCR apoya la enseñanza de la Lengua de Señas Costarricense (LESCO) a la comunidad y a cualquier persona interesada. Ofrece cursos organizados en lecciones, con cupo, matrícula y pago simulado. Sus usuarios son el **estudiante**, que consulta el catálogo y se matricula; el **instructor**, asignado a las lecciones; y el **administrador**, que publica cursos y aprueba pagos.

Proceso 1, matrícula y pago. Un estudiante elige un curso publicado y una lección. El sistema valida cupo, duplicados y referencia de pago, calcula el precio y guarda matrícula y pago en una sola transacción. El administrador aprueba el pago y la matrícula pasa a `ACTIVA`.

Proceso 2, publicación de un curso. El administrador publica un curso si cumple siete reglas, entre ellas fechas coherentes, ningún duplicado publicado, lecciones con instructor y al menos un recurso multimedia registrado en MongoDB.

2. Vista de contexto (C4, nivel 1)

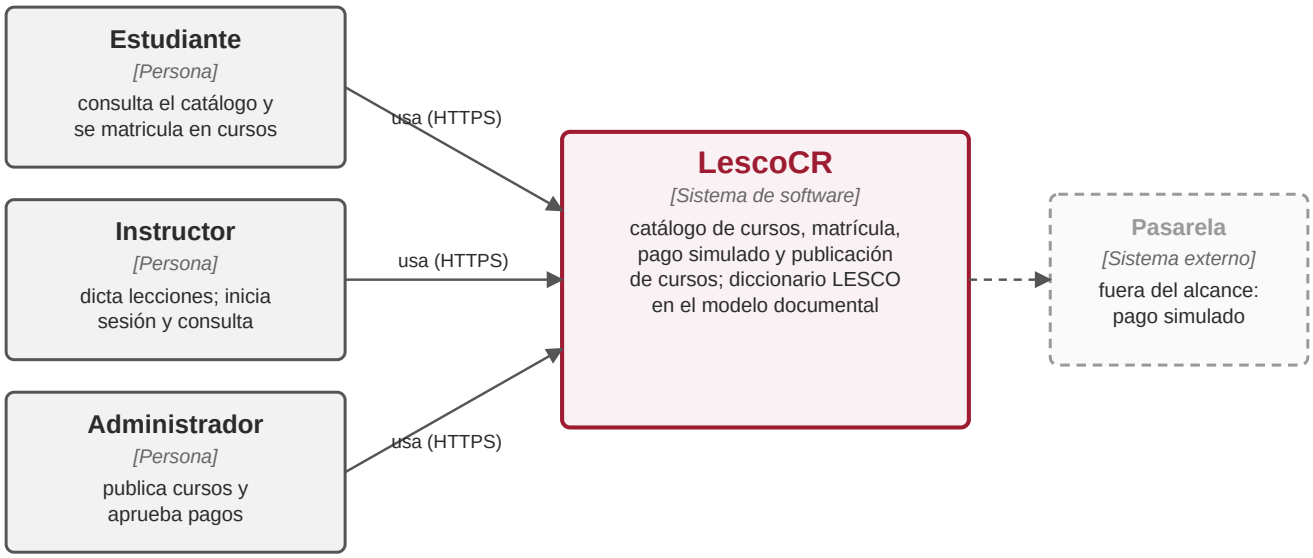


Figura 1. LescoCR y sus usuarios. El sistema no se integra con sistemas externos: el pago es simulado.

3. Vista de contenedores (C4, nivel 2)

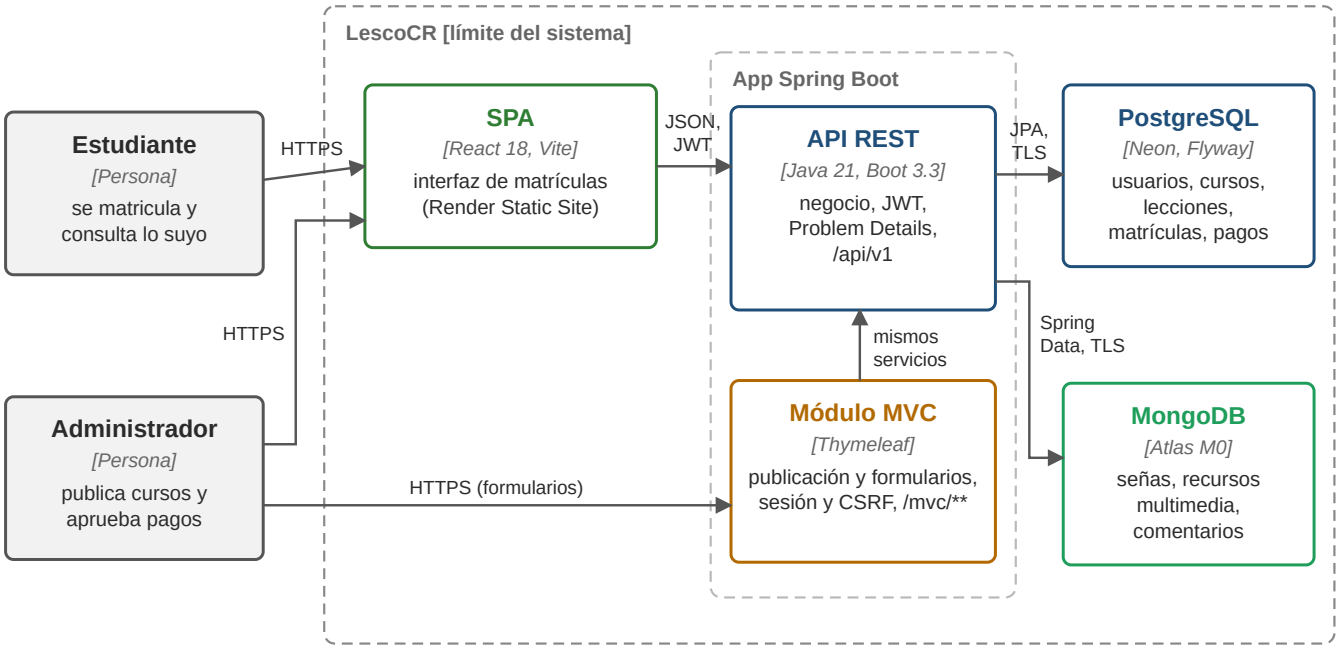


Figura 2. Lo que se despliega y cómo se comunica.

La SPA es el cliente principal del estudiante y envía JSON con un JWT a la API. El módulo MVC corre en la misma aplicación Spring Boot, usa sesión y CSRF, y reutiliza los mismos servicios de negocio, sin duplicar reglas. Solo esa aplicación accede a las bases de datos, siempre con TLS: PostgreSQL guarda los datos transaccionales con el esquema versionado por Flyway, y MongoDB guarda el contenido de estructura variable.

4. Vista de componentes (C4, nivel 3)

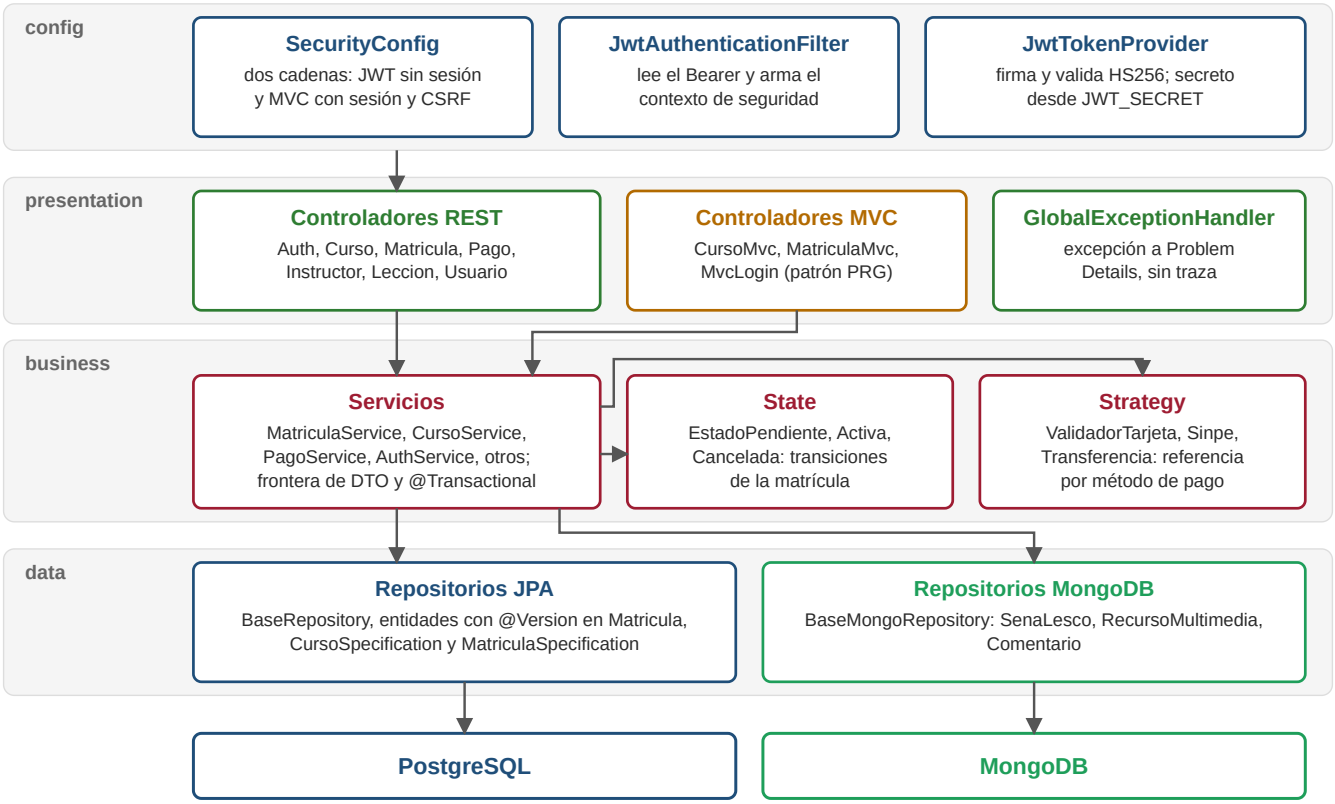


Figura 3. Elementos internos de la aplicación Spring Boot. Una petición baja de presentación a datos; ninguna entidad JPA sale de business.

5. Decisiones de arquitectura

Se registran las cinco decisiones tomadas durante el ciclo, con sus alternativas y sus costos.

ADR-001 · Dominio y alcance del sistema

Estado y fecha

Aceptada (agosto de 2026).

Contexto

El curso exige un sistema con al menos seis entidades, dos procesos de negocio y un subdominio documental. El equipo quería resolver una necesidad real: apoyar la enseñanza de LESCO a la comunidad y a cualquier persona interesada.

Decisión

Construir LescoCR con dos procesos de extremo a extremo: **matrícula y pago** de un curso, y **publicación** de un curso. El pago es simulado. Durante el desarrollo, cada lección quedó con su propio instructor y se priorizaron estos dos procesos.

Alternativas consideradas

- **Solo un diccionario gratuito:** era el núcleo social de la idea, pero no aportaba un proceso transaccional con cupo, pago y estados.
- **Incluir pasarela real, facturación electrónica y aplicación móvil:** se descartaron en la propuesta porque superaban lo que un equipo de dos personas podía terminar en un ciclo.

Consecuencias

- Se gana un núcleo completo y demostrable, con reglas que ejercitan transacciones, seguridad y concurrencia.
- **Costo:** parte de la propuesta no llegó a la interfaz. El diccionario solo existe en el modelo de datos, sin API; tampoco hay registro de usuarios ni gestión de contenido para el instructor.

ADR-002 · SQL o NoSQL para cada subdominio

Estado y fecha

Aceptada (septiembre de 2026).

Contexto

La matrícula necesita atomicidad y restricciones fuertes: cupo, unicidad y estados. El contenido multimedia, en cambio, tiene campos que varían según el tipo y puede crecer sin una cota clara. El curso pide además evaluar un almacén documental.

Decisión

Usar **PostgreSQL** para usuarios, instructores, cursos, lecciones, matrículas y pagos, y **MongoDB** para señas, recursos multimedia y comentarios. Se embebe lo que se lee con su padre y tiene crecimiento acotado; se referencia lo que crece sin cota o vive en otra base.

Alternativas consideradas

- **Todo en PostgreSQL con columnas JSONB:** viable y más simple de operar; se descartó porque el curso requería evaluar un almacén documental.
- **Todo en MongoDB:** se perdían las claves foráneas, los `CHECK` y el índice único parcial que protegen la matrícula.
- **Embeber los recursos en la lección:** no es posible, porque la lección es una fila relacional.

Consecuencias

- Los esquemas se ajustan a cada tipo de dato, y una regla de la publicación consulta MongoDB dentro del flujo de negocio.
- **Costo:** hay dos bases que operar y ninguna transacción las abarca. Nada garantiza que `leccionId` y `cursoId` de MongoDB existan en PostgreSQL. Hoy el negocio solo consume `recursos_multimedia`.

ADR-003 · Patrones de diseño aplicados

Estado y fecha

Aceptada (septiembre de 2026).

Contexto

El enum de estados no impedía transiciones ilegales: cualquier servicio podía activar una matrícula cancelada. Además, cada método de pago tiene un formato distinto de referencia, y esa validación depende de otro campo del DTO.

Decisión

Aplicar **State** al ciclo de vida de la matrícula, sin setter público de estado, y **Strategy** a la validación del pago. Se complementan con Repository genérico (`BaseRepository`) y Specification para filtros dinámicos.

Alternativas consideradas

- **Un `switch` en `MatriculaService`:** más corto, pero mezcla las reglas y crece con cada método de pago.
- **Bean Validation para la referencia:** no expresa bien una regla que cambia según `metodoPago`.
- **State con setter público:** fue la primera versión; se corrigió porque seguía permitiendo saltarse la validación.

Consecuencias

- La regla vive en un solo lugar y es imposible de omitir; un método de pago nuevo no modifica el servicio.
- **Costo:** hay más clases para pocas reglas. Agregar un método también exige tocar `PagoStrategyConfig` y una migración para `ck_pago_metodo`. Además, `data.Matricula` depende del paquete `business.state`.

ADR-004 · Estrategia de autenticación y autorización

Estado y fecha

Aceptada (octubre de 2026).

Contexto

La SPA se publica en otro dominio y consume una API REST. El módulo MVC, en cambio, usa formularios donde el navegador envía cookies solo. Además, un rol válido no basta: un estudiante no debe poder operar matrículas ajenas.

Decisión

Definir **dos `SecurityFilterChain`**: JWT HS256 sin sesión ni CSRF para la API, y sesión con formulario y CSRF para `/mvc/**`. Las contraseñas usan BCrypt. Los roles se validan en `SecurityConfig` y la propiedad del recurso, en los servicios.

Alternativas consideradas

- **Sesión para todo:** exigiría estado en el servidor y resolver cookies y CSRF entre el dominio de la SPA y el de la API.
- **JWT también para el MVC:** un formulario HTML no adjunta el encabezado `Authorization` por sí solo.
- **Secreto JWT con valor por defecto:** fue la primera versión; se eliminó porque quedaba público en Git y permitía firmar tokens válidos.

Consecuencias

- Cada módulo usa el mecanismo que le corresponde y comparten usuarios y credenciales. La API puede escalar sin sesión compartida.
- **Costo:** un token no se puede revocar durante sus 24 horas de vigencia, y la SPA lo guarda en `localStorage`, expuesto ante un XSS. Hay dos configuraciones de seguridad que mantener.

ADR-005 · Plataforma, stack y estrategia de despliegue

Estado y fecha

Aceptada (octubre de 2026). Amplía el ADR de stack de agosto de 2026.

Contexto

El sistema debe estar público con HTTPS, sin secretos en Git y a costo cero. El curso recomienda Java 21, Spring Boot, Gradle, PostgreSQL, MongoDB y React.

Decisión

Adoptar el stack recomendado. Desplegar la **API en Render con Docker**, la **SPA en Render Static Site**, PostgreSQL en **Neon** y MongoDB en **Atlas M0**. **GitHub Actions** corre `./gradlew build` en cada push y PR a `main`. El perfil `prod` no tiene valores de respaldo.

Alternativas consideradas

- **ASP.NET Core con Entity Framework y SQL Server:** se evaluó en los primeros borradores; se prefirió el stack recomendado, alineado con los laboratorios.
- **Vercel para la SPA:** quedó documentado como opción B; se prefirió concentrar la SPA y la API en un mismo proveedor.
- **Render PostgreSQL:** quedó documentado como opción, y el equipo utilizó Neon.

Consecuencias

- El despliegue es reproducible y, si falta una variable, la aplicación no arranca en lugar de iniciar con una configuración insegura.
- **Costo:** los planes gratuitos se suspenden por inactividad y agregan latencia al primer acceso. El `Dockerfile` compila con `-x test`, y Atlas admite conexiones desde `0.0.0.0/0` por la IP dinámica de Render.

6. Modelo de datos final

PostgreSQL: esquema relacional en 3FN, versionado con Flyway (V1 a V11)

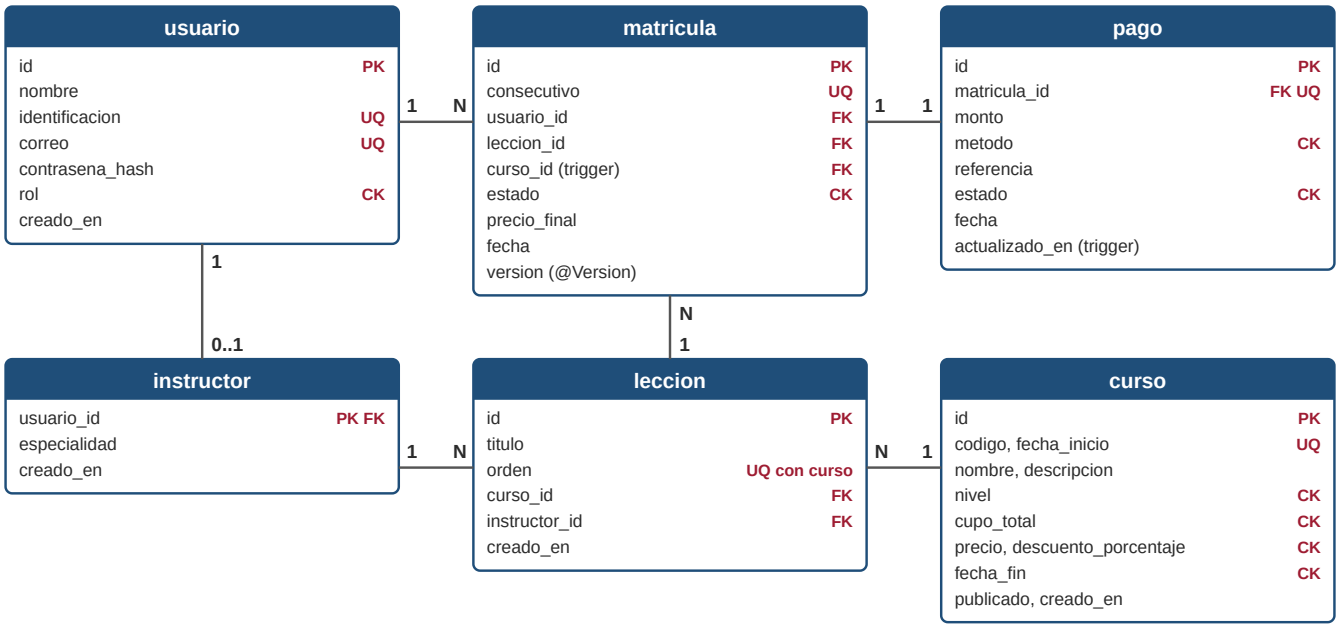


Figura 4. Diagrama entidad-relación. `matricula.curso_id` es una copia controlada de `leccion.curso_id`: la calcula un trigger para que el índice único parcial `uq_matricula_usuario_curso_vigente` impida dos matrículas vigentes en un curso.

Los cupos disponibles no se guardan: se calculan como `cupo_total` menos las matrículas `ACTIVA`. `precio_final` sí se guarda porque es un hecho histórico, no una copia del precio del curso.

MongoDB: colecciones con validador $jsonSchema

sena_lesco

```
{ palabra, palabraNormalizada,
  categoria: { codigo, nombre },
  multimedia: [{ tipo, url,
    esPrincipal }],
  tags: [...], activo }
```

Embebe categoría y multimedia: se leen con la seña y crecen poco.

recursos_multimedia

```
{ leccionId, cursoId,
  tipo, url, orden,
  metadata: {
    duracionSegundos,
    anchoPx, altoPx,
    tamanoBytes } }
```

Referencia lección y curso de PostgreSQL. Los campos de `metadata` varían según el tipo.

comentarios

```
{ cursoId, usuarioId,
  autorNombre, texto,
  calificacion, creadoEn,
  respuestas: [{ autorId,
    autorNombre, texto,
    creadoEn }] }
```

Referencia curso y usuario; **embebe** las respuestas, que no se consultan solas.

7. Estrategia de seguridad

Aspecto	Implementación
Autenticación	JWT HS256 de 24 horas para la API, sesión y CSRF para el MVC, y BCrypt para contraseñas. `JWT_SECRET` es obligatoria y no tiene valor por defecto.
Autorización	Roles en `SecurityConfig`. Propiedad del recurso en los servicios (OWASP API1): un estudiante recibe 403 ante una matrícula ajena, y en los listados se fuerza su propio `usuarioId`.
Defensas OWASP	Consultas parametrizadas con JPA y Specifications; DTO de salida; Problem Details sin traza; CORS por lista de orígenes; cabeceras por defecto de Spring Security; `/actuator/health` sin detalles; eventos de seguridad en logs JSON sin contraseñas ni tokens.
Concurrencia	`@Version` en `Matricula`; el conflicto responde 409 para evitar actualizaciones perdidas.

Resumen de la auditoría del Laboratorio 7

Corregido	Riesgo abierto
• Se eliminó el secreto JWT por defecto que estaba en el repositorio. • Se quitó el setter público de estado de `Matricula`. • Los errores inesperados ya no exponen mensajes ni trazas. • Las pruebas ya no se omiten en silencio si falta Docker.	• `GET /api/v1/instructores` es público y expone cédula y correo. • No hay límite de intentos de inicio de sesión ni CSP definida. • El token se guarda en `localStorage` y no se puede revocar. • Los errores MVC muestran el mensaje de cualquier `RuntimeException`.

8. Lecciones aprendidas

Qué funcionó. Probar contra PostgreSQL y MongoDB reales con Testcontainers detectó problemas que un mock oculta, como la validación del esquema. Fallar al arrancar si falta un secreto evitó configuraciones inseguras. Separar servicios y controladores permitió sumar el módulo MVC sin duplicar reglas. También aprendimos que una migración aplicada no se edita: el cambio va en una versión nueva (V9).

Qué cambiaríamos y por qué. Acotaríamos antes el alcance, porque el diccionario consumió trabajo de modelado y no llegó a la API. Protegeríamos el cupo con un bloqueo por curso, ya que `@Version` cubre cada matrícula pero no el total. Aplicaríamos las correcciones de N+1 a los listados de producción, no solo a las consultas de prueba. Agregaríamos pruebas del módulo MVC, hoy sin cobertura propia. Y redactaríamos cada ADR al tomar la decisión, para no reconstruir después sus razones.

Referencias

Bass, L., Clements, P. y Kazman, R. (2021). *Software Architecture in Practice* (4.ª ed.). Addison-Wesley.

Brown, S. (2018). *The C4 model for visualising software architecture*. c4model.com.

ISO/IEC/IEEE (2022). *ISO/IEC/IEEE 42010:2022. Software, systems and enterprise: Architecture description*.

Nygard, M. (2011). *Documenting Architecture Decisions*. cognitect.com/blog.

Richards, M. y Ford, N. (2020). *Fundamentals of Software Architecture*. O'Reilly.